

Hopfield model

Instructor: Mark Kramer

Today

Describe and simulate the Hopfield model

Original paper

Proc. Natl. Acad. Sci. USA
Vol. 79, pp. 2554–2558, April 1982
Biophysics

Neural networks and physical systems with emergent collective computational abilities

(associative memory/parallel processing/categorization/content-addressable memory/fail-soft devices)

J. J. HOPFIELD

Division of Chemistry and Biology, California Institute of Technology, Pasadena, California 91125; and Bell Laboratories, Murray Hill, New Jersey 07974

Contributed by John J. Hopfield, January 15, 1982

John J. Hopfield

Facts



© Nobel Prize Outreach.
Photo: Nanaka Adachi

John J. Hopfield
Nobel Prize in Physics 2024

Born: 15 July 1933, Chicago, IL, USA

Affiliation at the time of the award: Princeton
University, Princeton, NJ, USA

Prize motivation: “for foundational discoveries and
inventions that enable machine learning with artificial
neural networks”

Prize share: 1/2

Work

When we talk about artificial intelligence, we often mean machine learning using artificial neural networks. This technology was originally inspired by the structure of the brain. In an artificial neural network, the brain’s neurons are represented by nodes that have different values. In 1982, John Hopfield invented a network that uses a method for saving and recreating patterns. He found inspiration in physics’ models of how many small parts in a system affect the system as a whole. The invention became important in, for example, image analysis.

<https://www.nobelprize.org/prizes/physics/2024/hopfield/facts/>

Motivation

We design circuits to perform computational tasks

Evolution does not

... but, the brain performs computational tasks.

How?

“a spontaneous collective consequence of having a large number of interacting simple neurons.”

Motivation

A “simple” model of memory
... ignore biological details

Memory should

- work with partial information
- be robust to errors

Complete item

Hopfield JJ. Neural networks and physical systems with emergent collective computational abilities. Proc Natl Acad Sci U S A. 1982 Apr;79(8):2554-8.

Partial information

Hopfield Proc Natl Acad Sci. 1982

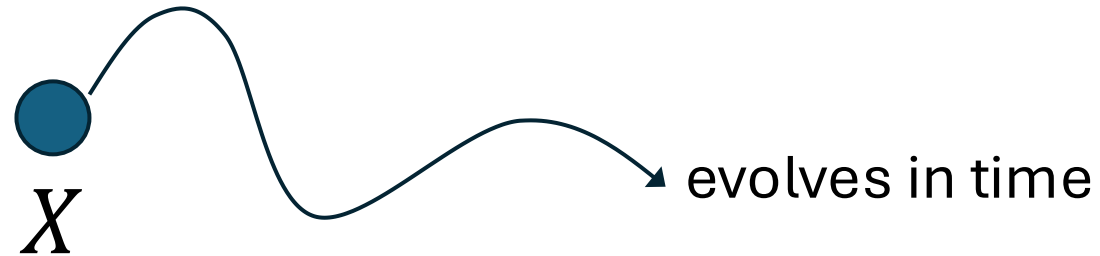
Partial information with error

Hopfield Proc Natl Acad Sci. 1983



Motivation

Physical systems can act to error correct

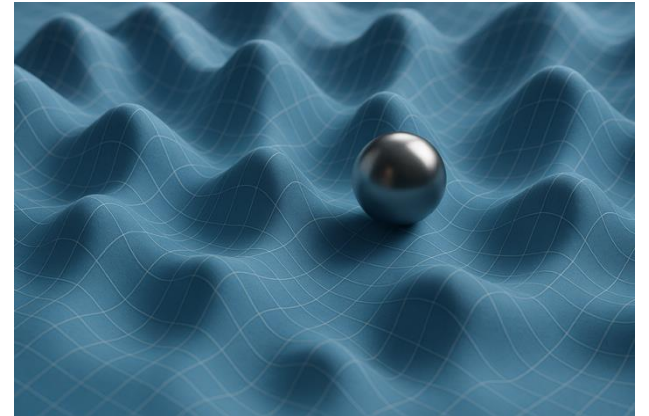


A physical system described by general coordinates X

Example: Marble rolling on a surface

1-dimension: position x ; velocity v

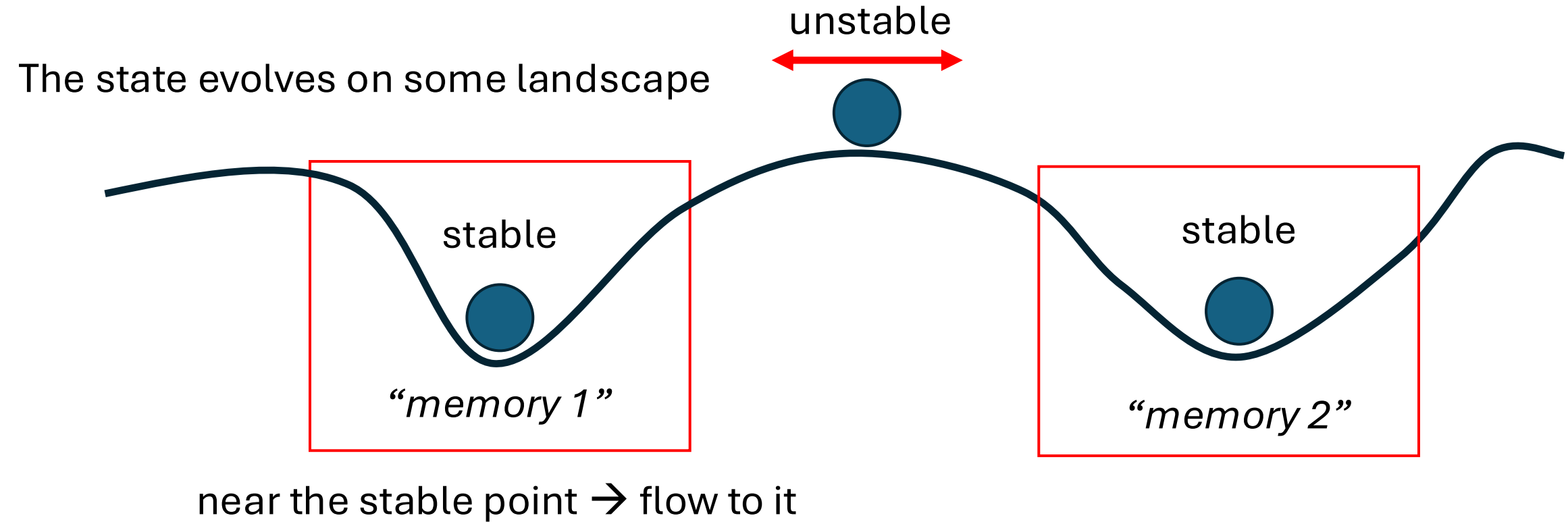
2-dimension: positions x, y ; velocities v_x, v_y



Motivation

X is a point in **state space**

memory $\rightarrow$ locally stable points



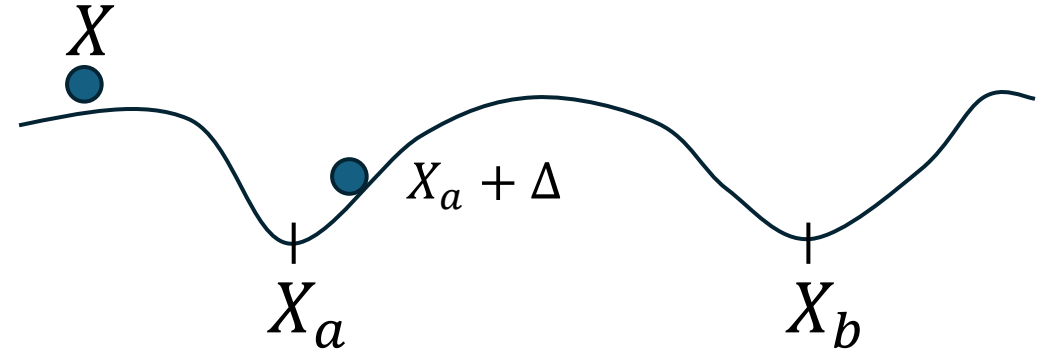
Motivation

X a point in state space

$X_a, X_b, \dots$ locally stable points

Start near a locally stable point

As time goes on



$X = X_a + \Delta$ partial knowledge of item

$X \approx X_a$ system approaches correct memory

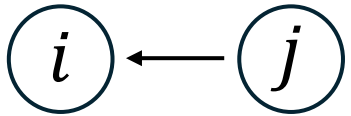
“Any physical system whose dynamics in [state] space is dominated by a substantial number of locally stable states to which it is attracted can therefore be regarded as a general content-addressable memory.”

With that motivation, now ... the model system

Model system

Consider neuron i

$$V_i = \begin{cases} 0 & \text{if neuron is "not firing"} \\ 1 & \text{if neuron is "firing at maximum rate"} \end{cases}$$

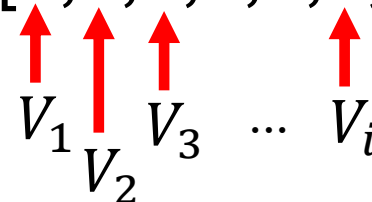


T_{ij} = connection to neuron i from neuron j
= 0 if not connected

Note: reciprocal connections are allowed

Q: What is T_{ij} ?

The state of a system with N neurons is a **binary word** $[0, 0, 1, 1, 1, 1, 0, 1, \dots]$



Model system

Dynamics: V_i (the state of neuron i) ... changes in time

$$\begin{array}{l} V_i \rightarrow 1 \\ V_i \rightarrow 0 \end{array} \quad \text{if} \quad \sum_{j \neq i} T_{ij} V_j \quad \begin{array}{l} > U_i \\ < U_i \end{array} \quad \begin{array}{l} U_i = \text{fixed threshold} \\ = 0 \end{array}$$

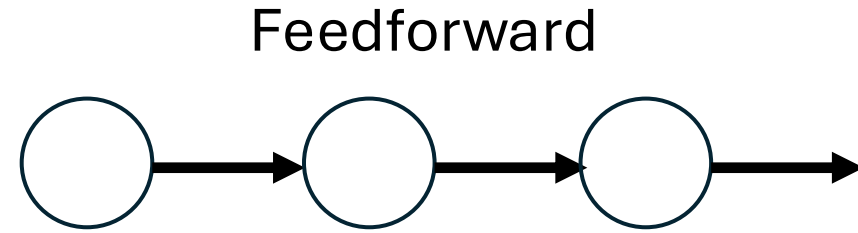
matrix

vector (we'll review linear algebra soon ...)

Each neuron i adjusts its state V_i randomly in time (at some rate)

Model system

Compare to the Perceptron



Perceptron

feedforward

sequential updates

Hopfield

allows backward coupling

random updates

Python

1. Simulate Hopfield network

Model system

Define a new variable

$$\underset{\substack{\uparrow \\ [-1, 1]}}{S} = 2 \underset{\substack{\uparrow \\ [0, 1]}}{V} - 1$$

$$\text{sign}\{x\} = \begin{cases} +1 & \text{if } x > 0 \\ -1 & \text{if } x \leq 0 \end{cases}$$

Define new dynamics

$$\begin{aligned} V_i &\rightarrow 1 & \text{if } \sum_{j \neq i} T_{ij} V_j &> U_i \\ V_i &\rightarrow 0 & \text{if } \sum_{j \neq i} T_{ij} V_j &< U_i \end{aligned}$$

Original Hopfield 1982

$$S_i = \text{sign} \left\{ \sum_{j \neq i} T_{ij} S_j \right\}$$

New (simpler) notation
[Kleinfeld 2024]

Q: What is T_{ij} ?

Information storage

We want to store a state

$$S = \xi$$

Ex:

$$\xi = [1, 1, -1, 1, 1, -1, 1, 1, 1, 1]$$

Storage prescription: **Choose T_{ij}**

Q: What is T_{ij} ?

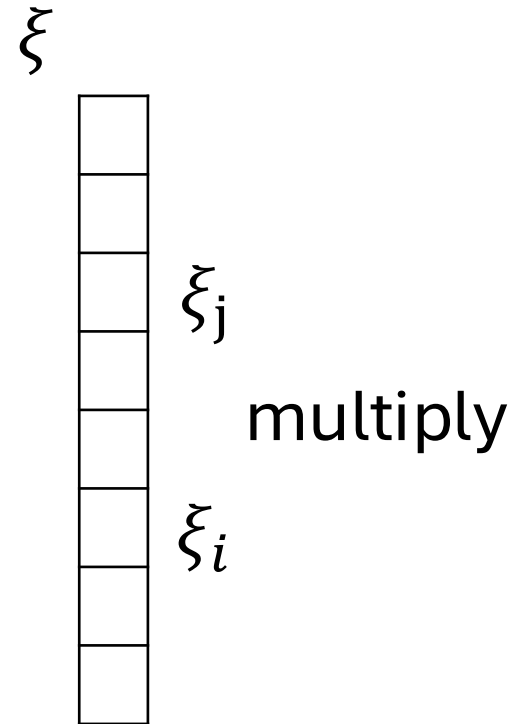
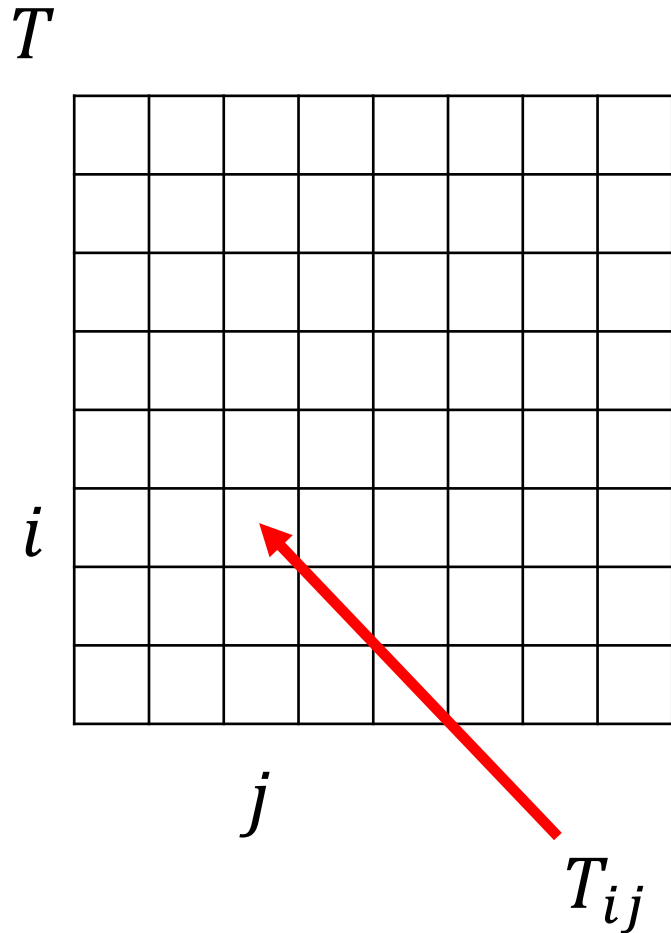
$$T_{ij} = \xi_i \xi_j \text{ where } T_{ii} = 0$$

What is this?

Information storage

What is this?

$$T_{ij} = \xi_i \xi_j$$



Why use this T_{ij} ?

Information storage

Case 1: Consider the update rule for the stored stable state ξ

How does ξ evolve due to dynamics?

update rule

$$S_i = \text{sign} \left\{ \sum_{j \neq i} T_{ij} S_j \right\}$$

$$\xi_i = \text{sign} \left\{ \sum_{j \neq i} T_{ij} \xi_j \right\}$$

$$\xi_i = \text{sign} \left\{ \sum_{j \neq i} (\xi_i \xi_j) \xi_j \right\} = \text{sign} \left\{ \sum_{j \neq i} \xi_i \xi_j^2 \right\}$$

$$= \text{sign} \left\{ \sum_{j \neq i} \xi_i \right\}$$

$\uparrow$
[(-1)², 1²] = 1

Information storage

Case 1: Consider the update rule for the stored stable state ξ

So

$$\xi_i = \text{sign} \left\{ \sum_{j \neq i} \xi_j \right\} = \text{sign} \left\{ \xi_i \sum_{j \neq i} 1 \right\} = \text{sign} \{ \xi_i (N - 1) \}$$

There are N neurons


$$\text{If } \xi_i = 1, \quad \text{sign} \{ \xi_i (N - 1) \} = 1$$

$$\text{If } \xi_i = -1, \quad \text{sign} \{ \xi_i (N - 1) \} = -1$$

So

$$\text{sign} \left\{ \sum_{j \neq i} T_{ij} \xi_j \right\} = \xi_i$$

→ the update rule does
not change the state ξ

It's **stable**

Information storage

Case 2: Consider the update rule for another state S (*not necessarily stable*)

How does S evolve due to dynamics?

$$S_i = \text{sign} \left\{ \sum_{j \neq i} T_{ij} S_j \right\} \quad T_{ij} \text{ stores the state } \xi$$

$$S_i = \text{sign} \left\{ \sum_{j \neq i} \xi_i \xi_j S_j \right\} \quad \text{What is this?}$$

Information storage

Case 2: Consider the update rule for another state S (*not necessarily stable*)

$$S_i = \text{sign} \left\{ \sum_{j \neq i} \xi_i \xi_j S_j \right\} \quad \text{What is this?}$$

Assume S mostly matches ξ

What is the product?

... mostly +1 with a few -1

→ sum > 0

$$S_i = \text{sign} \{ \xi_i \text{ (positive number)} \}$$

Information storage

Case 2: Consider the update rule for another state S (*not necessarily stable*)

$$S_i = \text{sign}\{ \xi_i \text{ (positive number)} \} = \text{sign}\{ \xi_i \}$$

↑
depends on the sign of ξ_i

So, $S_i \rightarrow \xi_i$ State S moving towards the stable, stored state ξ

This works and $S_i \rightarrow \xi_i$ as long as $\sum_{j \neq i} \xi_j S_j > 0$

i.e., S_j has at least 50% overlap with ξ_j

Information storage

Case 2: Consider the update rule for another state S

How does S evolve due to dynamics?

$$S_i = \text{sign} \left\{ \sum_{j \neq i} T_{ij} S_j \right\} \quad \text{update rule}$$

$$S_i = \text{sign} \left\{ \sum_{j \neq i} \xi_i \xi_j S_j \right\} \quad \text{What is this?}$$

Python

2. Pseudoorthogonality

Information storage

Consider

$$S_i = \text{sign} \left\{ \sum_{j \neq i} \xi_i \xi_j S_j \right\} = \text{sign} \left\{ \xi_i \underbrace{\sum_{j \neq i} \xi_j S_j}_{\text{What is this?}} \right\}$$

Conclusions

$$\sum_{j \neq i} \xi_j S_j$$

mean ≈ 0 when S & ξ are random (i.e., different patterns)

mean > 0 when S & ξ are similar (i.e., nearby patterns)

pseudoorthogonality

Aside: Linear Algebra